
SRE基础-1

如何理解SRE这个岗位？

我理解的 SRE，一就是致力于消除琐事。如我之前学习 TextFSM，就是因为我不希望每次巡检都人工登录设备去肉眼查看回显。我希望把这些时间节省下来，花更多精力去治理大规模集群或者优化架构

第二个就是：SRE 不是要打造一个 100% 不出故障的系统，而是用客观的数据作为裁判，在‘稳定性’和‘迭代速度’之间寻找平衡点，这种机制让运维和开发不再是对立的，而是协作

第三就是：在故障发生时，首要目标是止损（快速恢复业务，而不是查根因），而不是找人背锅，要防患于未然，通过监控提前发现问题，通过自动化减少人工操作，通过故障复盘优化系统，最终降低服务故障率

比如去设定一个服务的 SLO

先找 SLI：我会从用户视角出发，选择黄金指标（如成功率、延迟）。

设定阈值：不追求 100%。我会参考过去的性能数据，设定一个既能满足用户预期，又不会让团队疲于奔命的目标（99.9%）。

留缓冲：我会将 SLO 设得比 SLA 更严格（比如 SLA 是 3 个 9，我的 SLO 就是 3.5 个 9），这样在触发报警时，还有时间抢修，不至于直接赔钱

关于AIOps

AIOps 的本质是一个数据处理闭环：数据获取 -> 数据分析 -> 智能决策 -> 自动化执行

在实际场景中：

智能告警和噪声收敛：传统的阈值告警，可能因为产生抖动，造成告警风暴，利用聚类算法和拓扑关联，将同一时间段、同一链路上的相关告警合并为一个“根因事件”，向SRE发送一条高质量的聚合告警
这里举个简单例子----定一个滑动时间窗口（如 5 分钟），利用机器学习算法分析在这个时间段内爆发的大量告警，故障往往是瞬间爆发的。同一时间段内激增的告警，极大概率属于同一个根因。如果监控同时收到底层的 MySQL 出现大量慢查询和死锁。中间层的订单服务出现 HTTP 500 错误。最上层的网关报告 504 Gateway Timeout，通过算法发现它们都在同一条业务调用链上，从而将这数百条告警折叠成一个核心事件：“由于 MySQL 死锁导致订单链路整体不可用”

异常检测：算法通过学习历史时间序列数据，预测当前时刻的正常值，一旦实际数据偏离预测基线，即判定为异常，有利于提前预防

容量规划与智能扩缩容：基于历史流量趋势进行预测（例如预测几小时后的大促流量洪峰），提前触发 Kubernetes 的扩容动作，实现真正的“防患于未然”

当然，也可以使用agent的方式

云运维组件

Load Balancer

调度策略

算法名称	英文缩写	工作原理	最佳适用场景
轮询	RR (Round Robin)	按时间顺序将请求逐一分配给后端的服务器。	后端服务器硬件配置一致、处理能力相同的无状态应用。
加权轮询	WRR (Weighted RR)	在轮询的基础上为服务器配置权重。权重越高的服务器接收的请求越多。	后端服务器硬件配置存在差异，老机器权重低，新机器权重高。
最少连接数	LC (Least Connections)	统计后端各服务器当前的活跃连接数，将新请求分配给当前连接数最少的节点。	长连接业务（如 WebSocket），或请求处理时间差异较大的业务。
源 IP 哈希	Source IP Hash	对客户端的 IP 地址进行哈希计算，将结果与服务器数量取模，映射到固定的节点。	需要会话保持（Session Sticky）的业务，确保同一用户的请求始终落在同一服务器上，防止登录态丢失。
URL 哈希	URL Hash	对请求的 URL 路径（如 /video/123.mp4 ）进行哈希计算，映射到固定的节点。	缓存架构（如 CDN 或 Nginx 静态缓存）。确保相同的资源请求始终打到同一台机器，极大提升后端服务器的本地缓存命中率。

高可用与健康检查

负载均衡器周期性地向后端服务器发送探测请求，以检查其运行状态，当某台服务器连续多次探测失败，LB 会自动将其从后端可用池中剔除，停止分配新流量；当其恢复正常后，再自动加入池中

为避免负载均衡器自身成为单点故障，通常采用主备模式和集群模式

数据包转发模式

NAT模式：DNAT，SNAT，会出现性能瓶颈

DSR模式：不修改IP地址，但是会将数据包的MAC地址改成后端真实服务器的MAC地址，等到服务器将数据包处理完后，直接把响应包发送给客户端，不再经过LB（公用一个虚拟IP地址）

TUN 模式：如果后端服务器和 LB 不在同一个机房（跨地域跨网段），MAC 地址改不了。LB 就会像俄罗斯套娃一样，在原来的 IP 包外面，再套一层新的 IP 包头（目的 IP 指向异地服务器），通过隧道发过去。后端机器拆开外层包装，拿到真正的请求（就像是三层的DSR）

多级负载均衡架构

第一层（物理骨干网层）：用户的流量最先到达机房核心交换机，利用 BGP ECMP（等价多路径路由） 将流量均匀地打到下游的一组四层负载均衡集群上。
第二层（四层极速转发）：采用 LVS（DR 模式）。因为它工作在内核态，只负责改写 MAC 地址，不拆包，作为抗洪防线承受百万级并发，把流量分发给下游的七层代理。
第三层（七层业务网关）：采用 Nginx 或 Envoy 集群。在这里终结 HTTPS 证书，根据用户的 HTTP Header、URL 路径，把请求精准拆分到不同的后端微服务（比如交易系统、用户系统）。
第四层（微服务内部）：在 K8s 集群内部，服务之间的互相调用则由 Service (IPVS) 或 Istio (Envoy Sidecar) 接管，实现熔断和限流。

Nginx服务器

Master进程：不负责处理具体的网络请求，而是负责读取和评估配置文件，绑定端口，以及管Worker进程

Worker 进程：通常数量与服务器的 CPU 核心数绑定（`worker_processes auto;`）。它们是真正干活的进程，负责处理来自客户端的网络连接和请求。各个 Worker 进程之间相互独立，不需要加锁竞争资源，极大降低了上下文切换的开销

角色

当 Nginx 用来分发前端 HTML、图片、视频时，只要在配置里加上一句 `sendfile on;`，它就会调用内核的零拷贝技术，数据直接从磁盘 DMA 到网卡

用户永远看不到真正的后端（如 Python/Go 写的业务代码）。他们只能看到 Nginx 的 IP。Nginx 挡在最前面，不仅隐藏了内部网络拓扑，还能顺手把 HTTPS 证书的加解密给做了，卸载掉后端业务的计算压力

七层负载均衡

非阻塞、事件驱动机制

Nginx 的一个 Worker 进程就是一个单线程实体。它采用“多路复用”技术，在一个巨大的循环中监听数以万计的连接。当某个连接有数据可读或可写时，`epoll` 会通知 Worker 进程去处理；处理完一段逻辑后，如果遇到网络等待（如等待后端数据库返回），Worker 不会阻塞等待，而是立刻去处理下一个连接

反向代理和负载均衡

代理服务器位于真实服务器与客户端之间，对于客户端而言，Nginx 就是目标服务器，客户端对其发送请求；Nginx 接收请求后，再将其转发给隐藏在内网的真实后端服务器
提供七层负载均衡：能够深度解析 HTTP 报文，根据规则将流量精准路由至不同业务线的微服务集群

在 Nginx 层配置 SSL 证书，开启 HTTP/2 支持。公网流量在 Nginx 处完成解密，内网中 Nginx 与后端服务器之间走极速的明文 HTTP 协议，降低整体计算开销

Ansible自动化运维平台

基本原理

Ansible 最大的特点就是无代理（Agentless）。它不需要在被管理的服务器上安装任何客户端软件，而是通过 SSH（Linux/Unix）或 WinRM（Windows）与目标主机通信。这大大降低了被管理节点的维护成本，并消除了因代理程序带来的安全风险。

当你执行一个 Ansible 指令时，Ansible 会在控制机上动态生成一段临时的 Python 脚本，通过 SSH 把这段脚本推送到目标机器的 `/tmp` 目录下，执行完毕返回 JSON 结果，最后再把临时脚本删掉。

Playbook：是一个高度可读的 YAML 文件。你把需要执行的几十步操作（建用户、装 Docker、推配置文件、启动服务）按顺序写进去

Roles（角色化解耦）：当你的剧本越来越庞大时（比如你要完整交付一套包含 Prometheus、Grafana、Alertmanager 的监控栈），把所有代码塞在一个文件里会极其臃肿。Ansible 提供了 Role 机制，允许你把任务分解成独立的、可复用的模块。你可以专门编写一个名为 `monitoring_stack` 的 Role。这个 Role 内部有着极其严谨的目录结构：`tasks/` 负责环境安装和容器启动，`templates/` 负责分发动态配置文件，`handlers/` 负责在配置变更时触发服务重启

幂等性

Ansible 登录目标机器后，会先“看一眼”现状。如果 Docker 已经按要求跑着了，它直接返回绿色的 `OK`，什么都不做。只有发现状态不一致时，它才会执行操作，返回黄色的 `Changed`

幂等性意味着绝对的容错能力。在复杂的网络环境中部署几百台机器，如果中间网络断了导致部署失败，你不需要小心翼翼地排查哪些机器装了一半、哪些没装。你只需要闭着眼睛把刚才的 Ansible Playbook 重新再跑一遍。已经装好的机器会被直接跳过，失败的机器会继续推进，直到全网达到你期望的“最终一致性状态”

缺点

Ansible 默认并发数（`forks`）只有 **5**，如果真面对 1000 台机器，默认配置会极其缓慢，甚至引发 SSH 连接风暴导致控制节点内存打满

面试官问：你说 Ansible 是幂等的。但如果在你的部署过程中，某台机器因为网络抖动，`docker-compose up` 执行到一半断开了。你再次运行 Playbook，Ansible 怎么保证它不会重新拉起一套冲突的容器？

如果我用 `shell` 模块强行执行 `docker-compose up -d`，那它是没有幂等性的，每次跑都会触发执行。所以，我在项目中采用的是 Ansible 官方的 `docker_compose` 模块，Ansible 每次执行前，会先调用 Docker 的 API 去检查当前容器集群（Stack）的状态。只有当发现缺少容器或者配置有哈希变更时，才会触发真正的更新动作，从而实现绝对的幂等

其他工具：Terraform 不管机器里面的事，它管的是“向云厂商要资源”。你写一段代码，它直接帮你调用 API 购买 10 台 EC2 实例、配好 VPC、拉起负载均衡。如果环境坏了，不修，直接销毁重建。这叫不可变基础设施

横向对比

Ansible 和 Puppet、Chef 相比，最大的架构区别是什么？

答：Ansible 最大的优势是极简和无入侵。目标机器上不需要提前安装任何客户端，只要有 SSH 服务和 Python 运行环境即可。它采用 Push 模型，由控制节点主动发起连接

Ansible 底层到底是怎么执行任务的？你写的一个 `yum install`，远端机器是怎么认出来的？

答：当我运行 Playbook 时，Ansible 控制节点会先把 YAML 任务动态编译成一段临时的 Python 脚本。然后通过 SSH 登陆，把这个脚本传到目标机器的 `~/ansible/tmp` 目录下。接着通过 SSH 执行这段 Python 脚本完成实际操作，最后把脚本删除，清理案发现场，并将 JSON 格式的结果返回给控制节点。

Argo CD架构

作为一个k8s原生的控制器，直接部署在目标k8s集群

采用拉取模式，Argo CD 的底层运转围绕着两个极其核心的状态词：Target State（期望状态，Git 里的代码）和 Live State（实时状态，K8s 里的真实运行情况）

个人觉得很像MySQL里面，先写日志，再改数据

在 GitOps 的世界里，SRE 绝对不允许直接去敲命令修改生产环境。你想加一台服务器？你想修改 Nginx 配置？统统去改 Git 仓库里的 YAML 文件，然后提交一个 Pull Request (PR)。一旦代码合并，系统会自动把 Git 里的状态同步到线上

Git 仓库里写着“Nginx 版本 V2”，但集群里当前跑的是 V1。监工说：“不行，Git 是老规矩！”于是它主动在集群内部把版本升级到 V2

某个 SRE 偷偷登录集群，把原本 5 个容器删成了 2 个。监工一看：“不对啊！Git 仓库里写的明明是 5 个！”它会立刻触发警报，并强行把容器数量拉回到 5 个

Prometheus（普罗米修斯）

核心组件

HTTP Server：对外提供 API 服务。接收 Grafana 发来的查询请求，或者接收开发人员手敲的 PromQL 语句

Retrieval（抓取器）：负责根据配置，按照固定的时间间隔去各个目标节点主动“拉（Pull）”指标数据 TSDB，时序数据库

exporters，负责将某些不直接支持 Prometheus 协议的服务的监控指标，转换成 Prometheus 可以抓取的格式，也就是数据格式的转换

服务发现（SD） 让 Prometheus 具备了动态感知能力。它可以直接对接 Kubernetes API、Consul 或 AWS/华为云 API，自动获取当前存活的机器列表，然后自动去抓取

数据

Prometheus 保存所有数据均为时间序列，包括指标，标签，样本

指标说明了系统正在测量的物理量是什么，比如 `http_requests_total`

标签是附加在指标上的键值对，用于描述数据的多维特征，比如 `http_requests_total{method="GET", status="200"}`

样本是 Prometheus 每次去目标机器上拉取回来的真实物理数据点，`(1716420000, 1500)` -> 表示在 10:00:00 这个瞬间，数值是 1500

pull拉取模型

传统的监控大多是push：成千上万台服务器上的 Agent，主动把 CPU、内存数据打包，疯狂发给监控中心

Prometheus 服务器本质上就是一个极其强悍的定时 HTTP 爬虫。业务端（比如你写的 Python 程序）什么都不用管，只需要暴露出一个纯文本的 `http://IP:端口/metrics` 网页。Prometheus 会按照自己的节奏（比如每 15 秒），从容不迫地挨个去拉取数据

刚刚的argo cd也是这种模式

部署架构：**Argo CD vs Jenkins**（为了绝对的安全与防篡改）

Push：Jenkins 拿着最高权限的密码，强行把应用推进 K8s 集群。一旦 Jenkins 被黑，老家直接被端

Pull：Argo CD 坐在集群内部，每天主动去 Git 仓库拉取配置。外部系统根本进不来，集群绝对安全

监控架构：**Prometheus vs 传统监控**（为了保护监控大盘不被 DDoS）

Push：假设你有 1 万个微服务，如果它们同时向监控中心推送报警和日志数据，监控中心的网卡和 CPU 会瞬间被这种内部的“DDoS 攻击”打爆。

Pull：云原生时代的监控标准 Prometheus，绝对不接受别人主动发数据。它自己内部维护一个定时器，每隔 15 秒去各个微服务的暴露端口上拉取一次指标。

架构红利：监控中心完全掌握了主动权！如果监控服务器自己负载太高了，它只需要把拉取的频率从 15 秒改成 30 秒，瞬间就能给自己减负

消息队列：**Kafka vs RabbitMQ**（为了生死攸关的“背压 / Backpressure”）

Push：传统的消息中间件喜欢把上游发来的消息，立刻推给下游的消费者。如果上游是双十一洪峰（每秒 10 万条），而下游消费者的处理能力只有每秒 1 万条，下游的内存就会瞬间被塞满，直接 OOM死机！

Pull：现代大数据处理的王者 Kafka，采取的是极其冷酷的 Pull 模式。消息全存在 Kafka 的磁盘上，下游的消费者根据自己的消化能力，处理完一批，再去 Kafka 里拉取下一批

站在应用程序（比如 Nginx、Redis）的视角来看，**epoll** 也是一个极其经典的、纯正的 Pull（拉取）模式

时序数据库

优点

1. 先存内容后写磁盘的高并发写入
2. 使用二次差值的方法，压缩数据体积
3. 倒排索引，不需要进行全表扫描
4. 设置生命周期，滑动窗口维持数据体积

性能调优

面试官：如果集群规模很大，你这个Prometheus出现了瓶颈，怎么办？

1. 控制时间序列基数，避免高基数爆炸，设置权重，使用不同的采集周期
2. 横向扩展，引入部署多个下层的子 **Prometheus**，按照业务线（比如一组负责监控中间件如 **Redis/MySQL**，另一组负责业务容器）或物理机房进行功能分片，然后再部署一个全局的 **Global Prometheus**，仅仅去抓取下层叶子节点经过聚合后的关键指标，从而把抓取和计算的壓力分摊到多个下级节点
3. 引入 **Thanos** 或 **VictoriaMetrics** 这样的云原生解决方案

先看看监控数据的完整过程

宿主机的 CPU 状态实际上是由 Linux 内核实时统计，并动态暴露在宿主机的虚拟文件系统

`/proc/stat` 中的

Prometheus 根据配置的 `scrape_interval: 15s`，每隔 15 秒向 `node-exporter` 的 9100 端口发起一次 HTTP GET 请求，`node-exporter` 将内存中的指标转化为纯文本的 HTTP 响应，Prometheus 收到后，给该数据打上标签

数据落盘（先写日志后写盘）

Grafana 会向 Prometheus 接口发送一条 PromQL 查询语句，Prometheus 收到请求后，利用倒排索引迅速定位到这台机器的 `node_cpu` 时间序列。它从内存或磁盘中取出过去 5 分钟的采样点，在服务端完成 `rate()` 速率转换和平均值的数学计算，最后渲染成大屏

面试官：Prometheus如何面对高并发场景？

pull 的模型，根本上避免服务被海量数据压垮
内存优先的写入机制，极致的内存压缩（**tsdb**）
多集群架构

面试官：既然都是存数据，为什么不能把监控数据存进 MySQL 里？

因为监控数据是海量的、按时间线追加的纯数字。MySQL 的 **B+** 树索引在面对每秒几百万次的写入时，磁盘 **I/O** 会瞬间崩溃，**TSDB** 专为监控而生，它对数据采用极致压缩，能轻松扛住每秒数百万个数据点的写入

特点：

倒排索引，毫秒级返回

到达一定时间后数据落盘，只读不可修改，不需要锁机制，简化了并发控制

数据压缩，使用二次差分，压缩内存和磁盘空间

面试官问：突然某天 Prometheus 狂吃内存，随后疯狂触发 OOM Killer 不断重启，Grafana 彻底瘫痪。你觉得根本原因是什么？怎么抢修？

标签 (Label) 滥用导致的“高基数 (High Cardinality)”爆炸
这通常不是 Prometheus 代码的 Bug，而是业务开发在打点时乱加 Label 导致的。在时序数据库中，Label 的每一种唯一组合都会生成一个全新的时间序列。如果开发在 Label 里塞入了 user_id、client_ip 或者 URL-Token 等千变万化的动态值，时间序列的数量会呈指数级爆炸，瞬间撑爆 Prometheus 的内存倒排索引。

云原生内容

什么是云原生

微服务：把大系统拆散，变成一个个独立开发、独立部署的微服务，架构解耦，互不影响

容器化封装：不可变基础设施，解决环境不一致的问题

动态编排：动态声明式管理，自动扩缩容

持续交付：自动跑单元测试，自动打 Docker 镜像，自动推送到 K8s 集群

容器

概念：基于 Namespace (命名空间) 和 CGroup (控制组)，本质还是一个进程

namespace：进程隔离，文件系统隔离，网络隔离，进程间通信隔离等等

cgroup：CPU限制，内存限制，磁盘限制，网络流量限制

三、它们是如何联手打造容器的？

当你在 Kubernetes 中下发一个部署应用的 YAML 文件时，底层的容器运行时 (比如 containerd 或 runc) 其实就默默做了这两件事：

- 1 第一步 (画地为牢)：向内核申请创建一套全新的 Namespace，把业务进程塞进去。现在，业务进程拥有了独立的主机名、虚拟 IP 和 PID=1 的错觉。
- 2 第二步 (套上枷锁)：在宿主机的 /sys/fs/cgroup/ 目录下创建一个新的文件夹，把业务进程的真实 PID 写进去，并在文件里填上你在 YAML 里写的 CPU 和内存 Limits。

1. 交付标准，它解决了‘环境一致性’的痛点，把代码和运行环境打包在一起，实现一次构建，到处运行
2. 资源隔离，相比虚拟机，它实现了轻量级别的隔离，秒级启动，资源利用率更高
3. 微服务架构，各模块解耦，配合 k8s 编排，实现真正的自动化扩缩容和故障自愈

所以在 Cgroups 里：

建目录 = 向内核申请创建一个隔离组

写数字到配额文件 = 调参，设置 CPU 时间片或内存字节数

写数字到 tasks 文件 = 把特定进程拉进这个隔离组

面试官问：你的云原生可观测监控项目的思路是什么？

1. 部署了官方的 Node Exporter，利用 Prometheus 客户端库在内存中暴露 /metrics 接口，Prometheus 通过 Docker 内部的 DNS 机制 (比如自动解析 custom-exporter:8000) 去定时拉取数据。然后 Grafana 通过调用 Prometheus 的 API，将底层数据渲染成实时监控大屏
2. 我在 Prometheus 中配置了基于规则的告警阈值 (比如 CPU > 80%)。触发后，告警推送给 Alertmanager，由它进行分组和收敛，最后推送到接收端
3. 利用 Docker Compose 将整个复杂的微服务生态进行了容器化封装，基于 Ansible 的自动化部署剧本，完成多台服务器的一键部署
4. 我利用阿里云自带的工具在测试环境中人为注入 CPU 和内存飙升的故障，来验证是否告警正常

Docker 容器和传统的虚拟机到底有什么区别？为什么现在大家都爱用容器？

虚拟机跑一个 web 服务，你需要给它虚拟出一整套完整的硬件（CPU、内存），并且还要在里面安装一个极其庞大的完整操作系统

容器可以说是更轻量级的虚拟机，秒级启动，它只是利用了 Linux 内核底层的 Namespace（做环境隔离）和 Cgroups（做资源限制）技术，把进程圈了起来。容器将代码、运行时、系统工具、系统库都打包在一起。开发环境、测试环境、生产环境运行的镜像是一致的，彻底消除了环境差异导致的故障。

你在简历里写了用 Docker Compose 编排监控组件，那 Docker Compose 和 Kubernetes (K8s) 有什么区别？什么时候该用哪个？

Docker Compose 是一个极其轻量级的单机编排工具。我在做这个项目时，为了让 Prometheus、Grafana 等组件能在一台服务器上一键启停，且解决网络互通问题，写一个 docker-compose.yml 是最高效的

三大核心

不可变基础设施：如果这个容器出了故障，或者需要更新代码，云原生的做法是：直接把它杀掉（销毁），然后用最新的镜像重新拉起一个新的容器

声明式API：这就是云原生和 K8s 的灵魂。不再写步骤，只写一个 YAML 配置文件，告诉系统“我想要什么最终状态”

微服务架构与松耦合：云原生提倡将巨大的单体拆分成极小的**微服务**，支付是一个独立的容器，弹幕是一个独立的容器，它们各自独立部署，互不影响

说说k8s

Kubernetes 本质上是一个以 etcd 为唯一的单点事实，通过声明式 API 驱动底层无数个 Controller（控制循环），去不断调谐、抹平集群‘当前状态’与‘期望状态’差异的分布式资源调度操作系统。它利用底层容器技术的隔离能力和扁平的虚拟网络，屏蔽了底层数据中心的硬件差异

声明式 API：不关心过程，只关心结果，写的那份 YAML 文件，本质上是一份“**期望状态契约**”。把它交给 API Server，说：“我不管你用什么机器，我只要求全网必须时刻保持有 3 个 Nginx 副本在运行。

调谐循环：所有的 K8s 控制器内部全是一个无限的死循环，观察当前状态，对比期望状态，执行动作

面试官问：输入 kubectl apply -f nginx-deployment.yaml（创建一个 pod），k8s 内部都发生了什么？

客户端预处理：解析文件，转换格式，http 发送到 k8s 的 api server

api server 校验：校验，认证，将希望状态写入 etcd

Controller Manager: Deployment Controller 创建一个副本集 ReplicaSet，副本集对比期望与实际状态，创建 pod

kube-scheduler 打分，将 pod 与合适的 node 绑定

node 上的 kubelet 调用各个接口，运行容器，汇报状态

pod 状态的五个阶段：pending、running、succeeded、failed、unknown

容器的状态：ImagePullBackOff（镜像拉取失败），CrashLoopBackOff（再启动再崩溃）

K8s架构-控制平面

kube-apiserver: 它是整个 K8s 集群中唯一对外暴露接口的组件。无论是你敲下 `kubectl` 命令，还是集群内部其他组件互相通信，都必须先经过它的身份认证和权限审查

etcd: 一个分布式的、极其可靠的键值对（**Key-Value**）数据库

kube-controller-manager: 管理器，对比“期望状态”和“实际状态”。如果发现期望是 5 个 Nginx，但实际只有 4 个存活，它会立刻生成一个“创建 1 个新 Nginx”的任务，实现自愈功能

kube-scheduler: 当 **apiserver** 收到创建一个新应用的请求时，**scheduler** 会看一眼全厂的资源（哪台机器 CPU 空闲？哪台机器内存够大？），经过一套极其复杂的打分算法，最终决定把这个任务派发给哪台具体的 **worker** 节点

K8s架构-工作节点

kubelet: 听 **apiserver** 的命令。一旦 **scheduler** 把某个任务分配到了这台机器上，**kubelet** 就会立刻接旨，跑去仓库拉取 **Docker** 镜像，然后把容器启动起来。它还负责定时向大脑汇报本节点的心跳和资源使用情况

kube-proxy: 负责处理这个节点上所有进出的网络流量，它是 K8s 服务发现和四层负载均衡的执行者，负责把打到集群某个 IP 上的流量，精准地转发到具体的容器里

Container Runtime: 比如以前常用的 **Docker**，或者现在 K8s 默认的 **containerd**。它只负责最底层的苦力活：根据 **kubelet** 的指令，把容器的进程（**namespaces** 和 **cgroups**）隔离并运行起来

一份yaml文件有什么

顶层标识: `apiVersion` 与 `kind`

元数据: `metadata`

控制器期望状态: `spec`

模版定义: `spec.template`

容器具体规约: `spec.template.spec`

有状态和无状态的区别？

无状态: 服务器的内存里绝对不保存任何跨越多次请求的上下文数据。如果需要数据，要么让客户端每次在 HTTP Header 里自己带过来（比如 JWT Token），要么后端去集中式的数据库）里查。服务器本身只是一个“纯粹的计算通道”

有状态: 服务器把当前会话的数据存在了自己的本地内存或本地磁盘里。比如一个 WebSocket 长连接服务器，或者底层的 MySQL 数据库、消息队列等等

面试官问：怎么查看 K8s 里的 Pod 状态？如果 Pod 状态是 `CrashLoopBackOff`，你的排查思路是什么？

答：查 Pod 状态用 `kubectl get pod -n 命名空间`；排查 `CrashLoopBackOff` 的话，第一步用

`kubectl logs Pod名 -n 命名空间` 看容器启动日志，找报错原因；第二步用 `kubectl describe pod Pod名 -n 命名空间` 看 Pod 的事件信息，比如镜像拉取失败、资源不足、端口占用；第三步检查 Pod 的配置文件，比如镜像名称、挂载的存储、环境变量是否正确

istio-服务网格

本质还是解耦的思想，让业务人员可以专注于业务，网格之间的流量通信，检测，熔断，降级，负载均衡，都通过边车代理来实现，而且它本身，在控制和数据平面分离这一点，大有网络当中SDN的味道，当然也可以说是沿用了k8s的控制和工作节点分离

特点

流量管理：熔断降级，灰度发布，故障注入

灰度发布：按权重、HTTP header、来源 IP 等将流量切分到不同版本。

故障注入：主动注入延迟或 HTTP 错误码，测试服务韧性。

安全保障：

自动 mTLS：为网格内服务之间通信自动签发证书、加密、双向身份认证

授权等等

可观测性：

指标：自动生成丰富的 Prometheus 指标，也可以进行格式转换

除此之外，还有日志

无侵入性：通过 Sidecar (Envoy) 代理透明拦截流量，业务容器无需改动

Envoy 是作为 Sidecar 运行的，它是如何做到让业务代码完全感知不到它的存在的？（如何劫持的流量）

答：当一个 Pod 启动时，Istio 会注入一个名叫 `istio-init` 的初始化容器，这个容器拥有 `NET_ADMIN` 特权，它会在 Pod 所在的网络命名空间下发 iptable 规则，流量重定向

控制面 `istiod` 和数据面 `Envoy` 之间是怎么通信的？如果 `istiod` 宕机了，现有的业务流量会断吗？

答：如果控制面宕机，现有的业务流量绝对不会断，因为 Envoy 肚子里已经缓存了全量的路由表，只会导致“新的规则下发不下去”，不影响存量业务

网络

面试官问：在 K8s 里，Pod 之间为什么不能直接用 IP 地址互相通信？为什么非要引入一个 Service 概念？

如果一个 Node 节点宕机了，K8s 会在另一个节点上重新拉起这个 Pod，但它的 IP 地址会发生彻底的改变！如果前端服务直接把后端 Pod 的 IP 写死在代码里，一旦后端 Pod 重启，前端就彻底瘫痪了

Service 会提供一个永远固定不变的虚拟 IP (`ClusterIP`) 和内部 DNS 域名。前端服务只需要请求这个固定的名字，剩下的事它不用管

Service 背后会通过标签选择器 (`Label Selector`) 自动关联那一批真实存活的 Pod。当你请求 Service 时，它会在网络层 (TCP/UDP, 即四层) 通过轮询或随机算法，把流量转发给后端的 Pod 主要是处理东西流量

Ingress 解决了什么痛点？

管理员写一份 Ingress YAML，声明：‘把访问 `a.com` 的流量转给 `ServiceA`，把 `b.com` 转给 `ServiceB`’

外部流量只需要打到这一个 Nginx 上（只占用一个公网 IP 和 80/443 端口），Nginx 就会通过七层 HTTP 协议的 Host 头部和 Path 路径，精准地把流量反向代理到后端的各个 Service 上。主要是处理南北流量

总结（重要！）

总结一下，k8s 内部是一个“大平层”私网，里面的容器 IP 都是虚拟 IP，但是只要在 k8s 这个系统内，大家都认识，k8s 里面有一个规则系统，叫 `service`，并且它有一个完全虚拟的 IP（这个 IP 根本不存在，只是为了让容器发数据包的时候，可以有目标 IP 可以写），如果写入一条规则，规则中选择了标签，`endpoints` 会根据标签，找出所对应容器的 IP，每个 Node 上的 `kube-proxy` 会监听该 `endpoints` 的状态，当发现新规则后，让内核在 `IPVS` 里面写下相应的路由

Service 只知道“我要找贴着 `app=order` 标签的容器”，但底层真正干活的 `kube-proxy` 和 `IPVS` 根本不认识标签，它们只认识 IP。`Endpoints` 就是把“标签”翻译成“真实 IP 列表”的那个桥梁载体。

`endpoints` 就是 `endpoints Controller` 去监听 `api server` 写出来的，然后 `kube-proxy` 监听 `endpoints`

现在宿主机A下的容器1 要访问 宿主机B下的容器2的一份数据，而service有很多，cluster IP也不止一个，如何写目的IP呢？内部的虚拟DNS解析可以做到这个事情，现在目的IP有了，数据发出，到达宿主机网卡（docker0）后，宿主机的内核发现该IP是cluster IP，IPVS就通过一些算法，比如轮询，用真实的pod IP 替换该cluster IP，现在数据包在docker0这个虚拟网卡中，如果发现目的IP和源IP属于一个网段，docker0 直接实现二层转发，如果不在同一个网段，就要去另外一台宿主机B，这就到达真实世界了，那么，外面的真实世界是无法识别k8s内部的虚拟IP的，所以VXLAN就上场了，将数据再封装一次，那么现在源IP是该宿主机IP，如何得知目的IP呢？

当宿主机 B 刚刚加入 K8s 集群时，K8s 控制面就会给它分配一个专属的容器网段，比如 10.0.2.0/24。K8s 会把这个信息（“宿主机 B 的物理 IP 是 192.168.2.20，它管辖的容器网段是 10.0.2.0/24”）写进中心的 ETCD 数据库里，然后由flannel去封装处理，接着将包发过去

这里我想到BGP-EVPN也有类似功能，但是它是在传统数据中心网络中，在那里，容器不再是容器，而是真实的服务器，并利用叶脊结构，来实现快速转发

刚刚的属于东西流量，接下来说南北流量

发起请求：用户在手机浏览器输入 `https://www.example.com/api/data`

请求打到公网 DNS 服务器，解析出这个域名对应的是公司机房对外暴露的一个公网 VIP

LVS进行负载均衡，通过算法，将数据包发给后端的 Kubernetes 集群节点

在 K8s 中，为了让外网流量能进来，我们通常会建立一个特殊类型的 Service，叫做 NodePort。K8s 会在所有宿主机上强制开一个高端口，刚才的 LVS，实际上就是把数据包发到了某一台或几台宿主机的端口上。数据包刚进宿主机的真实网卡，宿主机的 Linux 内核有规则：凡是访问本机的 30443 端口的，统统 DNAT（目标地址转换）给（nginx是干活的，它的身份是Nginx Ingress Controller），nginx本身也是容器，它进行7层解析，Nginx 内部早就写好了 Ingress 规则写着：“符合这个域名和路径的请求，请交给内部的数据-service 处理，Nginx Ingress Controller本身会监听API server，也就是它不用找 cluster IP，直接找容器，把容器IP 作为目的IP，接下来又是东西流量的过程了

情景题

现在是周五晚上 8 点，抖音的一个核心活动上线。突然，你的 Prometheus 监控大盘一片爆红，Alertmanager 给你发了最高级别的 P0 告警。前端业务团队反馈，大量用户在请求我们用 Python 写的核心后端接口时，页面直接报错 504 Gateway Timeout。

你立刻通过命令行排查，发现 Kubernetes 集群里的 Python API Pod 状态全都是 Running（并没有崩溃或重启）。但是你把视线转移到底层的物理机（Worker Node）时，敲下 top 命令，看到了极其诡异的一幕：

这台机器是 16 核 CPU，CPU 的 us（用户态使用率）只有不到 10%，整体感觉 CPU 闲得发慌。，但是，系统 1 分钟的 Load Average 飙到了 85，CPU 的 wa（iowait 等待）指标极高

面试官问：既然 Pod 都在正常 Running，用户看到的 504 Gateway Timeout 到底是谁返回的？流量从用户手机，一路上经过了 K8s 的哪些网络组件才到达你的 Python Pod？

Pod 处于 Running 状态，只是说明容器没死、进程还在。但这不代表进程有能力处理请求。当用户的 HTTP 请求顺着 Ingress -> Service 打到 Python Pod 时，Python 迟迟不返回结果。Ingress 等待的时间超过了它配置的超时上限（比如 60 秒），于是 Ingress 掐断了连接，绝望地向用户返回了 504 Gateway Timeout

面试官问：结合你在 top 命令里看到的“低 CPU 使用率、极高的 Load Average、高 wa”这三个指标，请用底层原理给我翻译一下，这台物理机到底遭遇了什么瓶颈？里面的 Python 进程现在正处于什么状态？

cpu利用率低，load average高，iowait高，说明cpu的的压力并不大，它只是处于空转等待状态，因为load的计算除了cpu，还计算了阻塞进程的情况，此时说明磁盘的I/O很高，有进程正在一直读磁盘，python进程正处于阻塞状态

Linux核心

计算机体系结构

中断

硬件中断：这是外设（如网卡、磁盘、键盘）主动“打断”CPU的方式。比如，当网卡接收到一个网络数据包时，它会向CPU发送一个电信号。CPU必须立刻放下手中的活，保存现场，去执行网卡驱动中的中断处理程序

软件中断：这是软件主动触发的。最典型的的就是系统调用 (System Call)。当你的程序需要读取文件或发送网络请求时，没有权限直接操作硬件，只能通过触发软中断（如Linux下的 `int 0x80` 或 `syscall` 指令），主动陷入“内核态”，让操作系统代为执行

多核与并发

当操作系统决定剥夺当前线程的CPU使用权，换另一个线程上场，必须保存当前线程的“全部身家”（程序计数器、各种寄存器的状态），并加载新线程的状态。这个过程本身不产生任何业务价值，纯粹是管理开销。如果一个服务器上起了成千上万个不合理的线程，CPU的时间会全部浪费在切换上，表现为系统的 `sy` (内核态CPU使用率) 极高，但应用毫无吞吐量

为了避免线程在不同的CPU核心之间来回迁移（这会导致缓存全部失效），我们可以将特定的进程“绑”在特定的CPU核心上。在部署对延迟极其敏感的基础设施组件时，绑定核心是常规的调优操作

局部性原理

时间局部性：刚刚访问过的数据，很快会再次被访问（热数据，LRU算法）

空间局部性：刚刚访问过的数据，其内存地址附近的数据也很可能被访问。在处理数据结构时，比如遍历一个连续分配的数组，空间局部性极好，因为相邻元素会被打包加载到Cache的“缓存行 (Cache Line)”中（预读）

I/O模型

DMA 控制器芯片。CPU只需要给 **DMA** 下达一次宏观指令：“把网卡里的数据搬到内存的某个地址去，搬完叫我”。接下来的海量数据搬运工作完全由 **DMA** 硬件独立完成，不占用主CPU的一丝算力。搬运结束后，**DMA** 才会发一次中断通知 CPU 过来处理。这是现代高性能服务器不可或缺的底层机制

DPDK：完全绕过内核，网卡收到的数据，通过 **DMA** 直接映射到用户态内存中，操作系统内核连碰都碰不到

内核态 (Ring 0)：拥有最高权限，可以操作所有硬件、管理内存、调度进程。

用户态 (Ring 3)：权限受限，只能访问自己名下的内存。你写的业务代码、跑的 Nginx 或 MySQL 都在这里运行

常见 I/O 模型

同步阻塞：发起 `read` 后线程挂起，直到内核数据准备好并拷贝完成才返回，相当于我去买奶茶，站在柜台死等，什么都不干

同步非阻塞：发起 `read` 后立即返回 `EAGAIN`，用户不断轮询，直到数据就绪，再主动调用 `read` 拷贝数据，进程不断轮询，极其消耗 CPU

I/O 多路复用：`select / epoll` 阻塞等待多个 fd 中的任意一个就绪，然后用户再主动调用 `read` 拷贝数据（拷贝仍同步）

进程

R状态-正在干活 或 随时准备干活，如果用 `top` 看到有大量的进程处于 R 状态，并且这个数量远远超过了你服务器的 CPU 核心数，说明CPU 资源严重不足，系统很可能会卡顿

S状态-可中断睡眠，在等通知，可随时叫醒，是绝大多数服务的常态

D状态-不可中断，进程正在等待底层的I/O操作，比如读写本地磁盘，等待NFS网络文件系统返回的情况，如果系统 Load 飙升到 100，但 CPU 使用率只有 5%，这绝对是因为系统里出现了大量 D 状态进程。虽然它们不消耗 CPU，但 Linux 的 Load Average 统计算法是包含 D 状态进程的。这通常意味着底层存储发生了严重故障（如磁盘坏道、I/O 性能打满）

Z状态-僵尸进程，进程已经执行完毕退出了，它释放了所有的内存和资源，为了让它的父进程知道退出状态码是多少，内核会保留它的一个PID，正常情况下，父进程会立刻调用 `wait()` 或 `waitpid()` 把这个状态码取走，子进程就彻底消失了。但如果父进程代码写得烂（有 Bug），一直不去回收，这个PID就会一直卡在 Z 状态

经典问题：进程和线程的区别

调度与管理

进程是操作系统资源分配的基本单元；

线程是操作系统CPU调度执行的基本单元。

资源拥有

进程有独立的地址空间、内存、文件描述符等资源；

线程共享所属进程的所有资源，只私有栈、寄存器、程序计数器。

独立性

进程之间相互隔离，一个进程崩溃不影响其他进程；

线程共享资源，一个线程崩溃，整个进程都会崩溃。

切换开销

进程切换：需要切换页表、刷新 TLB，开销大；

线程切换：不换地址空间，开销远小于进程。

通信方式

进程通信复杂（管道、共享内存、消息队列等）；

线程可直接读写共享内存，通信简单高效。

虚拟内存

这是 Linux 最伟大的设计之一。每个进程启动时，都以为自己独占了整个系统的内存

虚拟内存必须映射到真实的物理内存 (RAM) 才能使用。这个映射关系存在“页表”里。CPU 里的硬件模块 MMU (内存管理单元) 负责在每次访问内存时，极其快速地将虚拟地址翻译成物理地址

当你用 `malloc` 申请内存时，内核只是在虚拟内存里给你画了个饼，并没有真的分配物理内存

直到你真正往这块内存里写数据时，CPU 发现这块虚拟内存还没有对应的物理内存，就会触发一次缺页中断。

内核这才赶紧去物理内存里找一块空地（Page），把它和虚拟地址映射起来。

如果物理内存不够了，内核会把一些不常用的内存页写到磁盘的 `Swap` 区里，这就叫换出（Swap out），极其影响性能

OOM Killer 机制

输入一条命令，linux里面发生了什么，比如ls

1. 回车后，键盘向CPU发送中断信号，CPU把字符读进缓冲区
2. 进行解析，识别，翻阅环境变量里的目录，找到对应的可执行文件/usr/bin/ls
3. 进程调用fork()，创建了一个子进程
4. 子进程执行调用，清空自己，加载可执行文件的代码，成为一个真正的ls进程，开始执行main函数
5. 向内核申请查看目录，VFS（虚拟文件系统）发力，将信息读取，返还给ls进程
6. 格式化输出，然后自杀，唤醒父进程

零拷贝

先看看传统，read() 和 write()的过程

DMA 拷贝：磁盘数据 → 内核缓冲区（page cache）

CPU 拷贝：内核缓冲区 → 用户内存

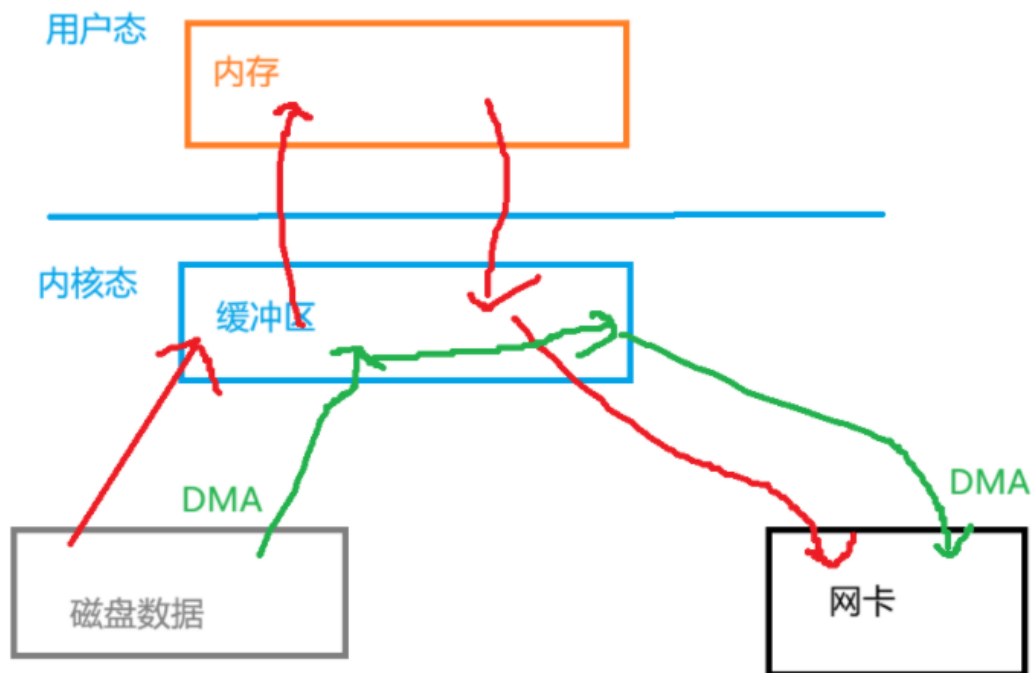
CPU 拷贝：用户内存 → Socket 内核缓冲区

DMA 拷贝：Socket 内核缓冲区 → 网卡

四次拷贝，四次上下文切换

内核只做了一件微小的事：它把数据在 Page Cache 里的文件描述符和长度等信息（元数据）追加到了 Socket 缓冲区。

网卡的 DMA 控制器直接根据这些元数据，顺藤摸瓜找到 Page Cache 里的真实数据，一把捞走发送到网卡



CPU拷贝0次，DMA拷贝2次，上下文切换2次

命令

系统资源与性能监控

top #实时查看系统负载，进程CPU/内存占用

iostat #详细监控磁盘 I/O 性能，查看磁盘利用率（%util）和响应时间（await）

进程管理

ps aux #列出当前所有进程。常配合 **grep** 查找特定服务

pidstat #统计指定进程的CPU、内存、IO消耗

网络排查

netstat -tulnp / **ss -tulnp** #查看当前系统开启了哪些端口，以及由哪个进程监听

tcpdump #抓包

lsof -i:80 #查看哪个进程占用了80端口

lsof: 作用非常纯粹，列出当前系统里所有被打开的文件，以及是哪个进程打开了它们

文件与磁盘排查

df -h #查看磁盘分区的挂载点及容量使用情况

ls -li #查看文件的inode信息

获取当前系统中的 TCP 连接数

ss -ant | wc -l

ss -s

dmesg 查日志的，层级最低，不受用户态影响，是 Linux 内核的“原始日志”，它能看到所有用户态工具看不见的底层事件